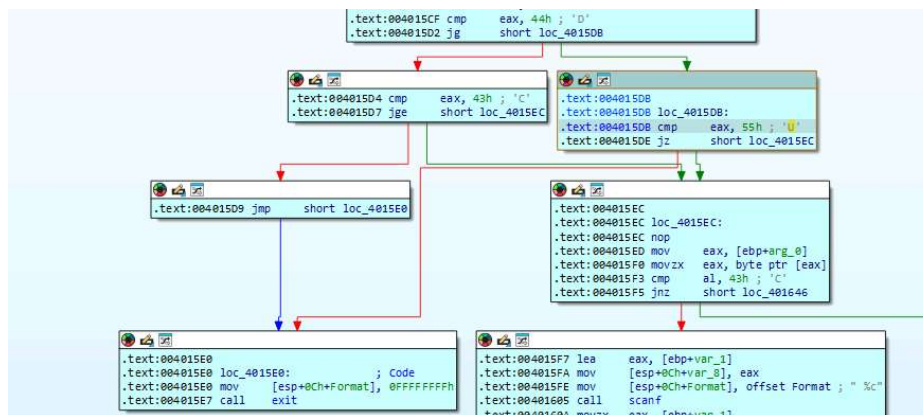


We had one last safe to analyze. Like the previous ones, it contained only a single file: safe.exe. As always, our goal was to reverse-engineer it and determine the correct input.

We began as usual by searching for user input handling, specifically looking for scanf calls. Surprisingly, the program had only two calls to scanf, both within the same function (40159C).

.text:004015B0	sub_40159C	call scanf
.text:00401605	sub_40159C	call scanf

From static analysis, we learned that the function first reads a character, which must be either C, U, or D—any other input causes the program to exit.



If the character is C, an additional numeric character is read.

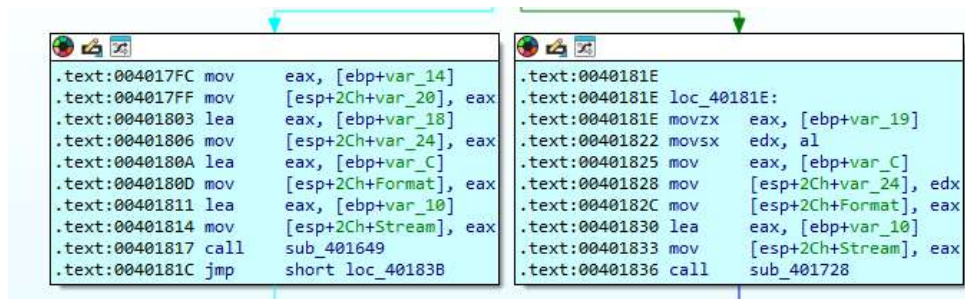
```
.text:004015F7 lea    eax, [ebp+var_1]
.text:004015FA mov     [esp+0Ch+var_8], eax
.text:004015FE mov     [esp+0Ch+Format], offset Format ; "%c"
.text:00401605 call    scanf
.text:0040160A movzx   eax, [ebp+var_1]
.text:0040160E movsx   eax, al
.text:00401611 mov     [esp+0Ch+Format], eax
.text:00401614 call    sub_401410
.text:00401619 movzx   eax, [ebp+var_1]
.text:0040161D cmp     al, 2Fh ; '/'
.text:0040161F jle     short loc_401629
```

At the end of the input process, the function stores the command and its associated number (in numeric form) in the variables it received via pointers.

At this point, the allowed inputs were still ambiguous, so we investigated the function

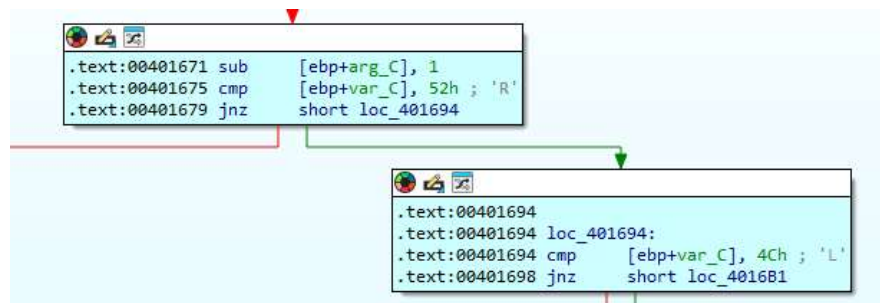
that calls the input routine (*sub_4017A7*). We'll refer to this as the main routine. After returning from the input routine, the program takes one of two paths:

- One for the case of C with a number (we'll call this the left routine),
- Another for U or D (the right routine).



The left routine receives the input number and three variables by reference from the main function. The right routine receives the character and two of the variables. This suggests that the left routine modifies all three variables, while the right one changes only the last.

At the beginning of the left routine, there is a check to see if the third variable holds the value L or R. Since U and D are already present, we infer these letters indicate directions: Up, Down, Left, Right. The third variable holds the horizontal direction.



At the start of the right routine, the program accesses a specific memory region, repeatedly referenced throughout the code. The memory access follows this formula:
 $base + var_1 * 8 + var_2$

```

.text:0040172E mov     eax, [ebp+arg_8]
.text:00401731 mov     [ebp+var_C], al
.text:00401734 mov     eax, [ebp+arg_0]
.text:00401737 mov     eax, [eax]
.text:00401739 mov     [ebp+var_4], eax
.text:0040173C mov     eax, [ebp+var_4]
.text:0040173F lea     edx, ds:0[eax*8]
.text:00401746 mov     eax, [ebp+arg_4]
.text:00401749 add     eax, edx
.text:0040174B add     eax, offset unk_408020
.text:00401750 movzx   eax, byte ptr [eax]
.text:00401753 mov     [ebp+var_5], al
.text:00401756 cmp     [ebp+var_5], 2Eh ; '.'

```

This resembles access to an 8-column 2D array starting at base. We tested this hypothesis and found that elsewhere in the code, a condition checks whether the two indices are within 0–7, confirming that this is indeed an 8x8 array:

- $var_1 = \text{row index}$
- $var_2 = \text{column index}$

We noted that the base array is dynamically initialized before the first input read. So we ran the program in debug mode, observed the array after initialization, and visualized it as an 8x8 matrix:

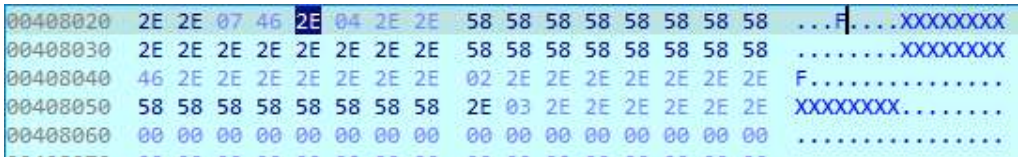


Figure 1: mem. table

.	.	7	F	.	4	.	.
X	X	X	X	X	X	X	X
.	.	.	.	.	.	.	.
X	X	X	X	X	X	X	X
F	.	.	.	.	.	.	.
2	.	.	.	.	.	.	.
X	X	X	X	X	X	X	X

.	3	.	.	.	.	.	.
---	---	---	---	---	---	---	---

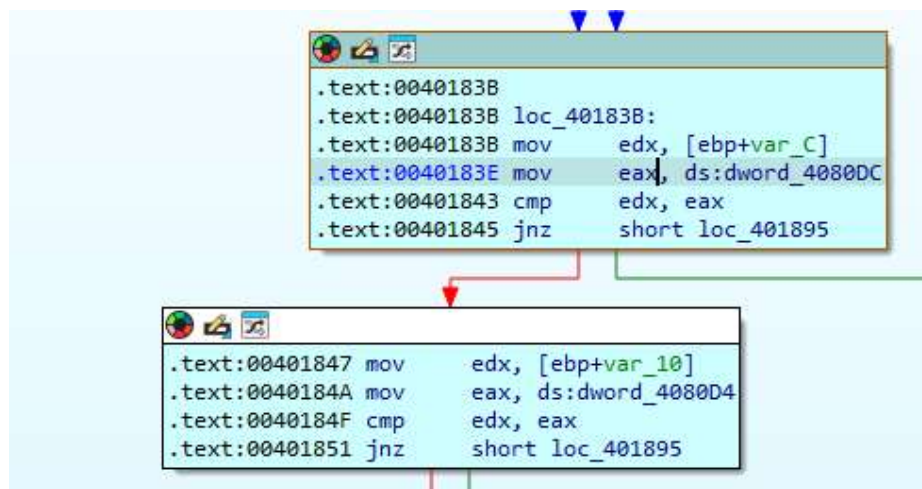
Table 1: reconstructed table

This clearly resembles a map, and based on previous findings, we realized the program simulates a game with directional movement.

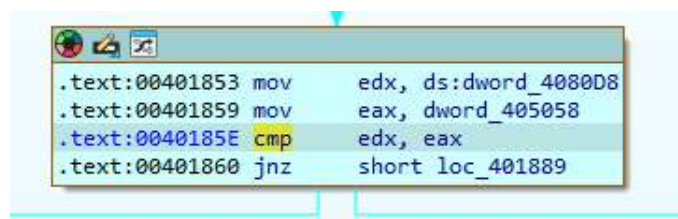
From code inspection, we saw the game starts with: (0,7).

That is, the starting position is at the top-right corner.

At the end of the main routine, a check determines if x and y have reached (7,7)—indicating the bottom-right corner. So, the goal is to reach the lower-right corner.



There is also a check for a global variable equal to 2.



We found this value increases only when the player reaches a cell containing F in the left routine.

```

.text:004016F6 mov     eax, ds:dword_4080D8
.text:004016FB add     eax, 1
.text:004016FE mov     ds:dword_4080D8, eax

```

When this happens, the F is removed and a score counter is incremented. This means the player must collect two Fs along the way.

Game Rules:

Left Routine (C command):

The direction (L or R) is stored in a variable.

$C(k)$ performs the move k times (recursion):

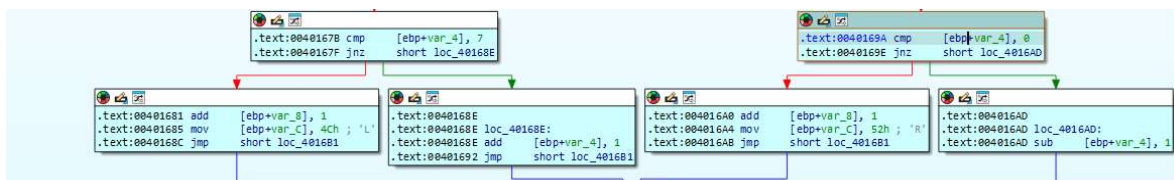
If direction is L, decrement x .

If direction is R, increment x .

If x exceeds bounds (0–7), x is not changed, but:

y is incremented (i.e., go down a row),

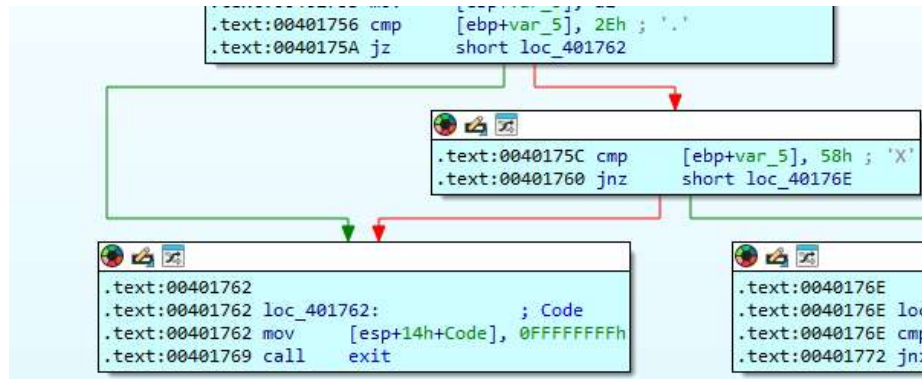
Direction is reversed.



This means C lets us move horizontally with wrapping to the next row if we hit the edge.

Right Routine (U/D command):

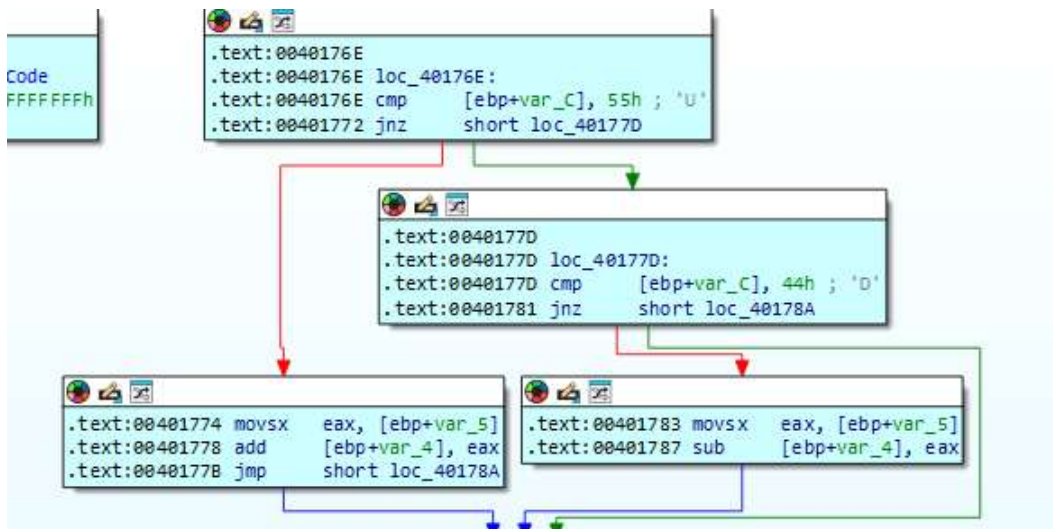
Allowed only on number or F cells.



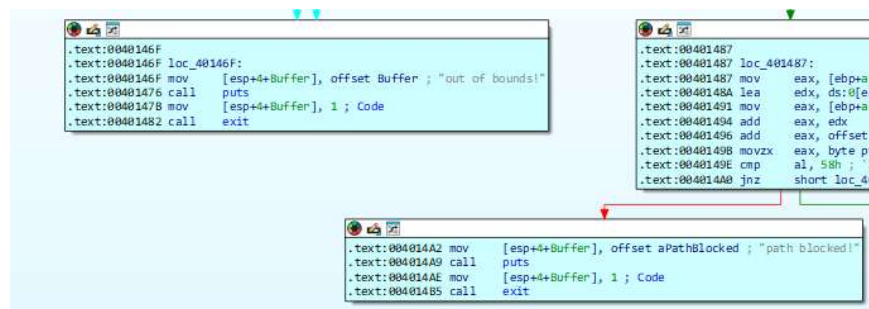
If current cell is a number and:

Command is U, the number is added to y.

Command is D, the number is subtracted from y.



If resulting cell is out of bounds or an X, the move is invalid.



Strategy and Solution:

Now that we understand the mechanics, we can determine the optimal path:

Start: top-right ($x = 0, y = 7$)

Goal: bottom-right ($x = 7, y = 7$)

Collect all Fs (two in total).

Using C commands for horizontal movement and U/D commands for vertical jumps (based on cell numbers), we map out the path.

In the visual diagram:

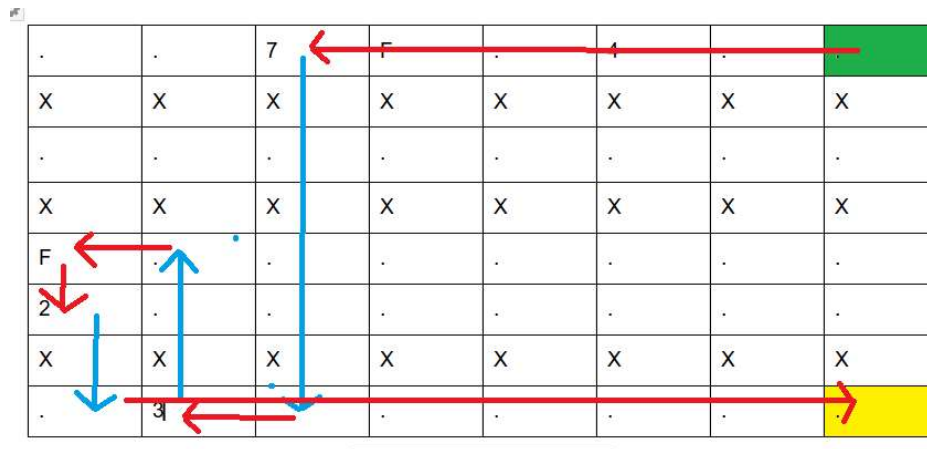
Green = Start

Yellow = End

Red arrows = horizontal C commands

Blue arrows = vertical U/D commands

We encountered a wall in row 4, so we changed direction accordingly.



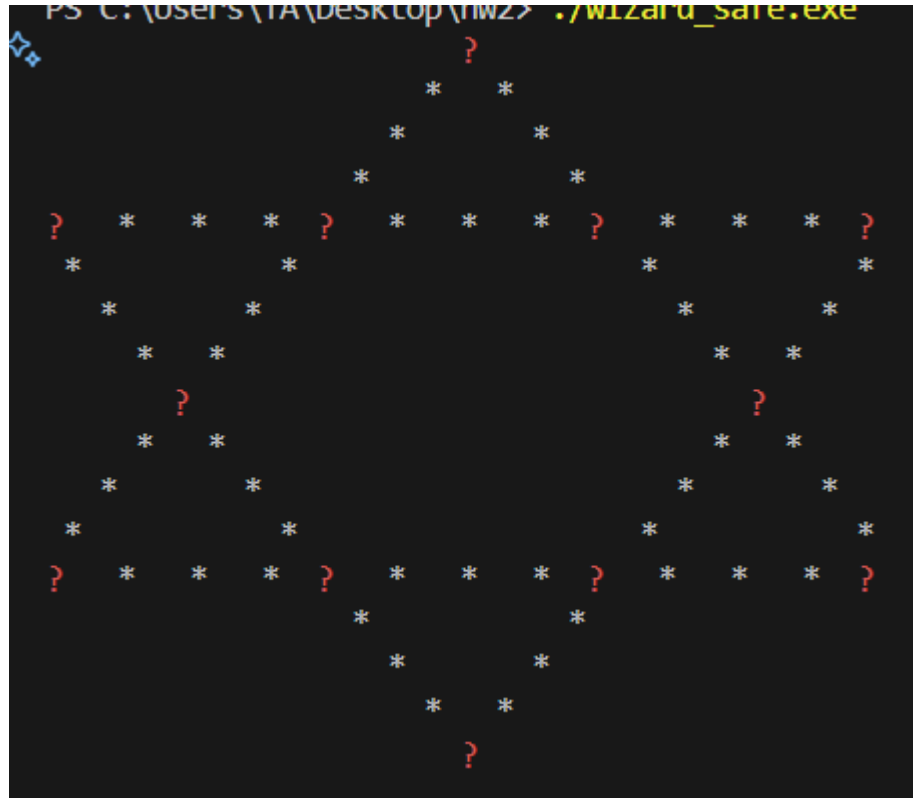
The input sequence that wins the game is: `C 5 U C 1 D C 2 U C 7`.

```
C 5 U C 1 D C 2 U C 7
The encryption super secret key is 6!%b\+l2c%C,9VK]
```

The key is `6!%b\+l2c%C,9VK]`.

Wizard.exe

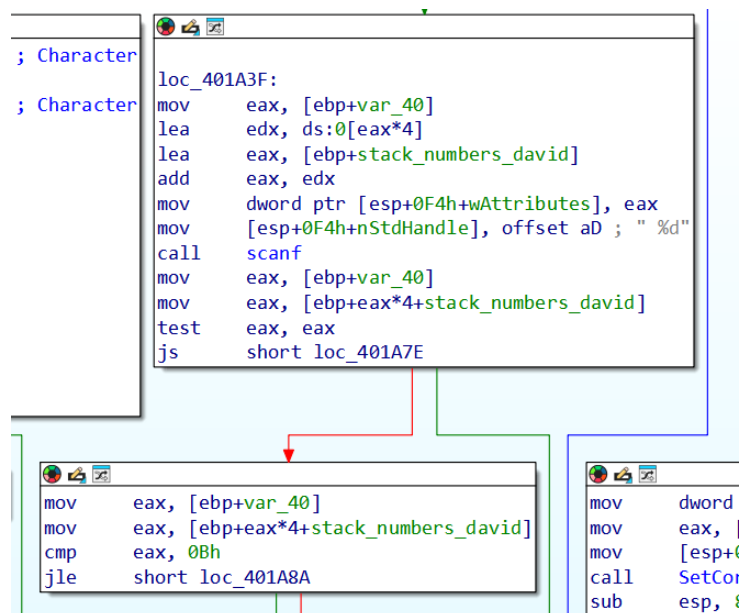
הרצנו את wizard וקיבלנו מגן דוד.



הנחנו שמדובר בחידה ואחרי גיגול מצאנו שצריך שסכום כל שורה או אלכסון יהיה 26.

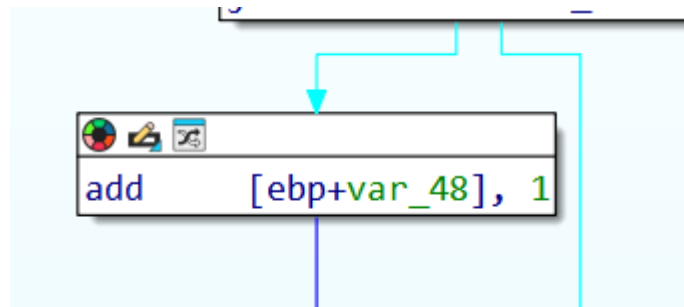
ניסינו להכניס מספרים מ 1 עד 12 כפי שצוין בחידה בגוגל וגילינו שזה לא עובד אז נכנסו לida.

מצאנו קריאה ל scanf:



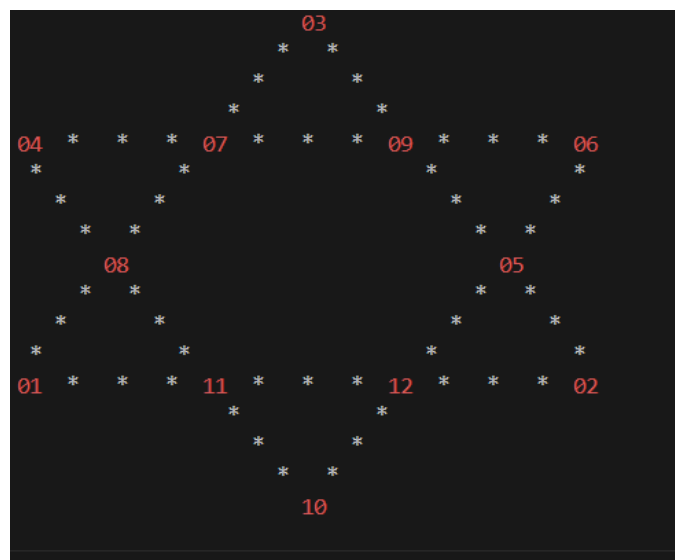
וראינו שיש השוואה ל B (12) כלומר כנראה שאנחנו בכיוון הנכון אבל שמנו לב ש אם מוכנס 12 נקרא exit ולכן צריך מספר קטן מ 12.

מצאנו שבהמשך(אם מכניסים מספר קטן מ 12) יש הוספה של 1:



ולכן הסקנו שצריך להכניס מספרים בין 0 ל 11.

אחרי הכנסה חוקית של מספרים הבנו שאכן כך(הודפס מגן דוד אמנם לא חוקי):

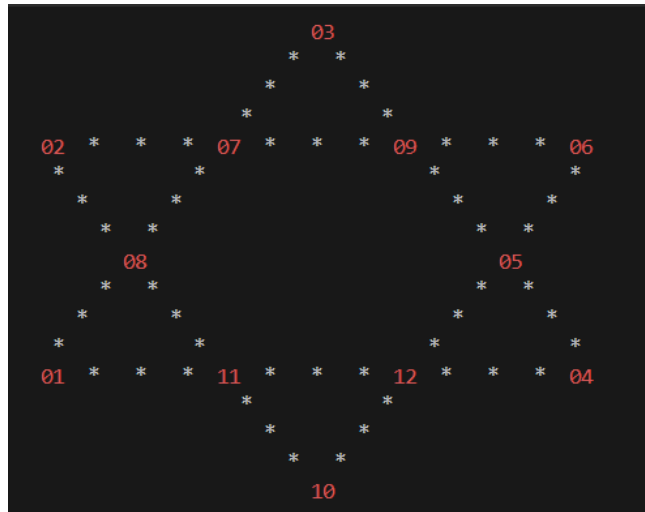


שמנו לב שהמספרים שהכנסנו ממוקמים במקומות "ראנדומליים" וניסינו לשחק עם המיקומים.

שמנו לב שאם הופכים את סדר הקלט לצורך העניין:

1 9 3 7 11 5 0 6 10 2 8 (המספרים שמייצגים את המגן דוד לעיל)

אם נהפוך את 9 ו 2 נקבל:



כלומר כל המגן דוד השתנה למעט 2 ספרות שהפכו מקום (2 ו 4 בשמאל למעלה וימין למטה).
לכן הסקנו שלפי המיקום בקלט הספרה ממוקמת במגן דוד, ולפי הספרה שהוכנסה מתקבלת ספרה חדשה.

ביצענו מיפוי ידני לכל הספרות (אחרי ניסיון לעשות brute force ב python שרץ יותר מדי זמן אז זנחנו את הרעיון) על ידי היפוכן עם ספרה שרירותית וההנחה שהתבררה כנכונה שהספרות ממוקמות מלמעלה למטה משמאל לימין במגן דוד בהתאמה לקלט וקיבלנו:

```

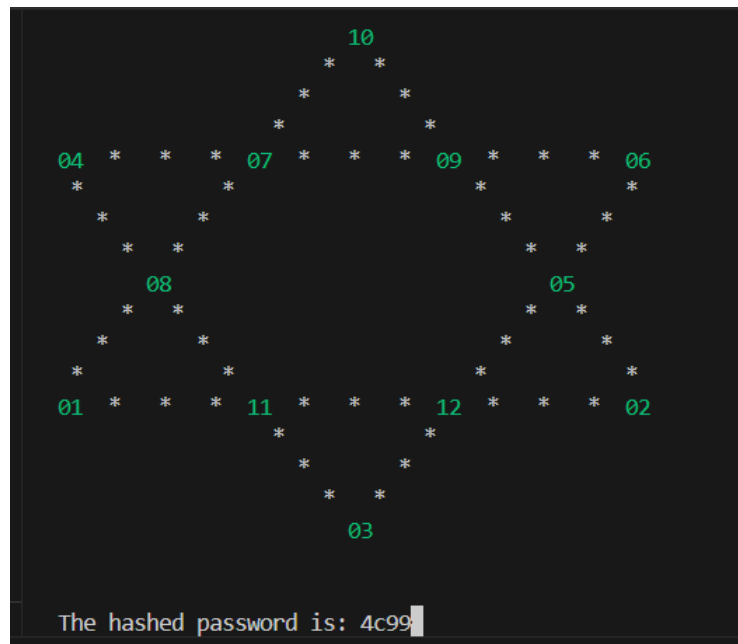
5->1
4->7
3->9
2->2
11->8
8->10
6->11
7->6
0->5
1->3
10->12
9->4

```

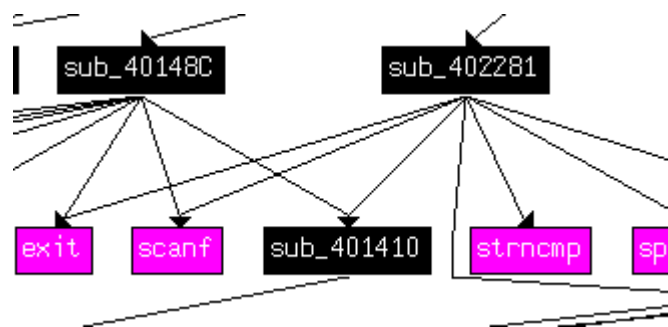
כלומר אם נכניס 5 נקבל את הספרה 1 וכו.
אחרי סידור הקלט שיתאים למגן דוד עם כמה פתרונות אפשריים מצאנו את הפתרון שהתאים עם הקלט:

8 9 4 3 7 11 0 5 6 10 2 1

עד שקיבלנו: hash password 4c99



ראינו שיש עוד scanf ולכן חיפשנו בגרף פונקציות מי קורא ל scanf:



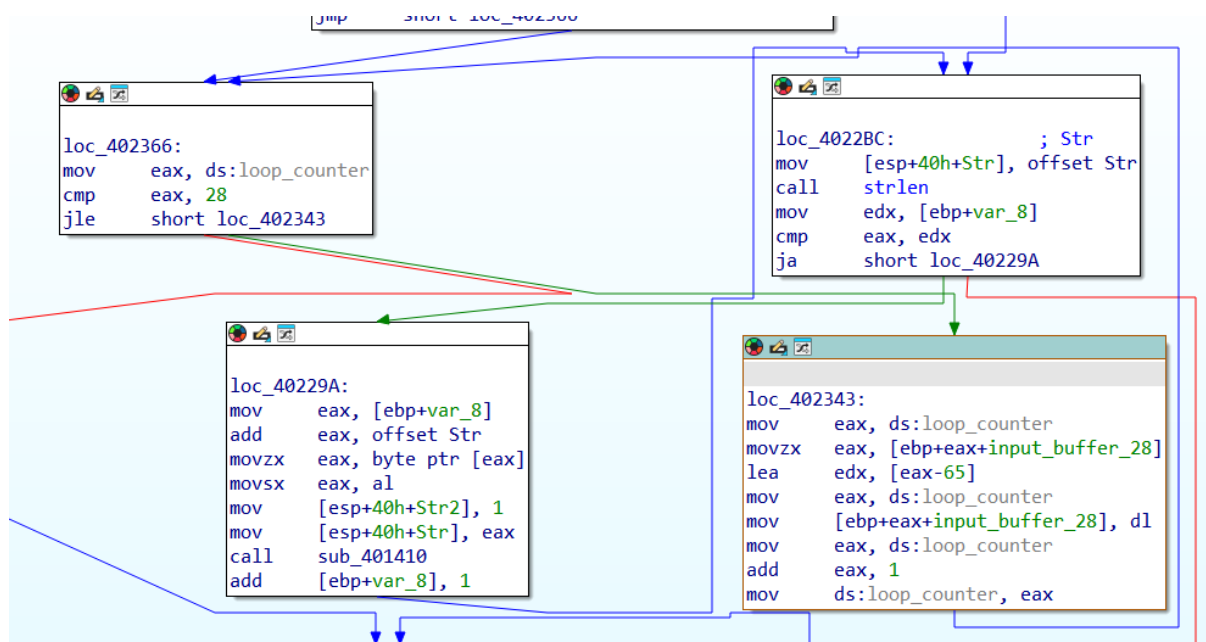
מצאנו בפונקציה ערך מחרוזתי חשוד:

```

mov     [esp+40h+MaxCount], 29 ; MaxCount
mov     [esp+40h+Str2], offset Str2 ; "DDLILMENBHLDDBBLOJOGAKHEKBDLI"
lea     eax, [ebp+input_buffer_28]
mov     [esp+40h+Str], eax ; Str1
call    strcmp
test    eax, eax
jnz     short loc_40248D
  
```

לאחר ניתוח סטטי הגענו למסקנה שמחרוזת שנכנסת ב scanf עוברת פרמוטציה ואז משווית לערך במחרוזת עבור 29 התווים הראשונים.

קיבלנו אישור להנחה כאשר ראינו לופ אחרי scanf שעושה פרמוטציה 29 פעם של הורדת 65 מכל תו:



לאחר מכן מצאנו עוד פרמוטציה:

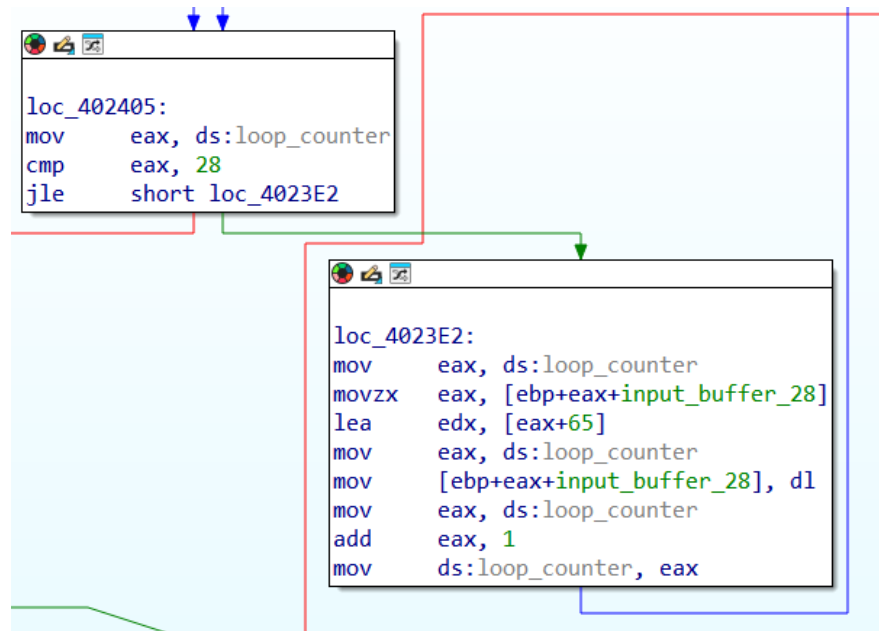
```

loc_40237C:
mov     eax, ds:loop_counter
movzx   ebx, [ebp+eax+input_buffer_28]
mov     eax, ds:loop_counter
lea     ecx, [eax+1]
mov     edx, 8D3DCB09h
mov     eax, ecx
imul    edx
lea     eax, [edx+ecx]
sar     eax, 4
mov     edx, eax
mov     eax, ecx
sar     eax, 1Fh
sub     edx, eax
mov     eax, edx
imul    eax, 1Dh
sub     ecx, eax
mov     eax, ecx
movzx   edx, [ebp+eax+input_buffer_28]
mov     eax, ds:loop_counter
xor     edx, ebx
mov     [ebp+eax+input_buffer_28], dl
mov     eax, ds:loop_counter
add     eax, 1
mov     ds:loop_counter, eax

```

שמנו לב למספר ה"חשוד" 8D3DCB09h והנחנו שמדובר ב magic number כלשהו שמשמש לביצוע חילוק / מודולו בלי ביצוע Div ולאחר מכן ראינו שיש הכפלה ב 1D(=29), כמו כן יש טיפול ב sign extention עם SAR על ערך 31 ולכן הנחנו שמתבצע Mod29(ולא div). לאחר המשיך ניתוח הבנו שמתבצע xor בין האות הנוכחית שנבדקת ל אות הבאה בתור ב Loop around ל 29 אותיות הראשונות בקלט.

לאחר כל התהליך שוב מתבצעת הוספה של 65:



ולכן כל שנותר לעשות הוא להנדס את הקלט ככה ש 29 התוים הראשונים לאחר xor יביאו את הפלט הרצוי "DDLILMENBHLDDBBLOJOGAKHEKBDLI".

קיבלנו שהקלט המתאים הוא:

.FGFOGNBFIJOFGEFOAJHBBLMICDAL

לאחר מכן יש קריאה ל sprintf.

```

mov     eax, offset sub_402281
mov     [ebp+var_2E], al
mov     eax, offset sub_402281
sar     eax, 8
mov     [ebp+var_2D], al
mov     eax, offset sub_402281
sar     eax, 16
mov     [ebp+var_2C], al
mov     eax, offset sub_402281
shr     eax, 24
mov     [ebp+var_2B], al
mov     [ebp+var_2A], 0
lea     eax, [ebp+var_2E]
mov     [esp+40h+var_34], eax
lea     eax, [ebp+input_buffer_28]
mov     [esp+40h+MaxCount], eax
mov     [esp+40h+Str2], offset aSS ; "%S%S"
lea     eax, [ebp+input_buffer_28]
mov     [esp+40h+Str], eax ; Buffer
call    sprintf
mov     eax, [ebp+4]
mov     [ebp+var_C], eax
cmp     [ebp+var_C], offset sub_402281
jz      short loc_4024A5

```

ונראה שצריך לדאוג שב ebp+4 יהיה offset sub_402281.

נשים לב שהכתובת של sub_402281 מוכנסת לsprintf בייט אחרי בייט החל מהקלט האחרון לכן אם נכניס מספיק תווים אחרי הקלט הרצוי עבור ה scanf שבודק 29 תווים בבדיקה הקודמת, אז נשתול את הפונקציה בדיוק במקום של ebp+4.

אחרי ספירה של הבתים הגענו למסקנה שצריך

$$60FF24 - 60FF14 = 10h = 16_{decimal}$$

כלומר עוד 16 תווים אחרי הקלט שהכנסנו ל scanf (כתוספת לקלט הזה) כדי להגיע למצב שה sprintf ישתול את הכתובת 402281 בכתובת ebp+4 וההשוואה תעבור.

המחסנית:

Stack[00032FF4]:0060FF13	db	49h	; I
Stack[00032FF4]:0060FF14	db	41h	; A
Stack[00032FF4]:0060FF15	db	41h	; A
Stack[00032FF4]:0060FF16	db	41h	; A
Stack[00032FF4]:0060FF17	db	41h	; A
Stack[00032FF4]:0060FF18	db	41h	; A
Stack[00032FF4]:0060FF19	db	41h	; A
Stack[00032FF4]:0060FF1A	db	41h	; A
Stack[00032FF4]:0060FF1B	db	41h	; A
Stack[00032FF4]:0060FF1C	db	41h	; A
Stack[00032FF4]:0060FF1D	db	41h	; A
Stack[00032FF4]:0060FF1E	db	41h	; A
Stack[00032FF4]:0060FF1F	db	41h	; A
Stack[00032FF4]:0060FF20	db	41h	; A
Stack[00032FF4]:0060FF21	db	41h	; A
Stack[00032FF4]:0060FF22	db	41h	; A
Stack[00032FF4]:0060FF23	db	41h	; A
Stack[00032FF4]:0060FF24	db	81h	

EBP+4:

.text:00402477	call	sprintf	
.text:0040247C	mov	eax, [ebp+4]	
.text:0040247F	mov	[ebp+var_C], eax	
.text:00402482	cmp	[ebp+var_C], [ebp+4]	
.text:00402489	jz	short loc_402281	
[ebp+4]=[Stack[00032FF4]:0060FF24]			
	db	81h	
	db	22h	; "
	db	40h	; @
	db	0	
	db	80h	
	db	0FFh	
	db	60h	; `
	db	0	
	db	33h	; 3
	db	12h	
_402281+1FB (Synchronized with EIP)			
1 F8 08 88 45	"@..E4.."@.....E		
4 B8 81 22 40	b". "@.....EC.."@"		
D 45 D2 89 44	E..E...E..D		

וסוף הקלט של ה Scanf לפני הוספה של padding:


```
Stack[00032FF4]:0060FF11 db 44h ; D
Stack[00032FF4]:0060FF12 db 4Ch ; L
Stack[00032FF4]:0060FF13 db 49h ; I
Stack[00032FF4]:0060FF14 db 41h ; A
Stack[00032FF4]:0060FF15 db 41h ; A
```

ולכן סהכ הקלט שנדרשנו להכניס הוא:

FGFOGNBFIJOFGFEOF AJHBLMICDALAAAAAAAAAAAAAAAA

כאשר במקום ה AAAA בסוף המחרוזת יכלנו להכניס כל 16 תוים אחרים שהם אותיות גדולות באנגלית.

וכך קיבלנו את הפלט:

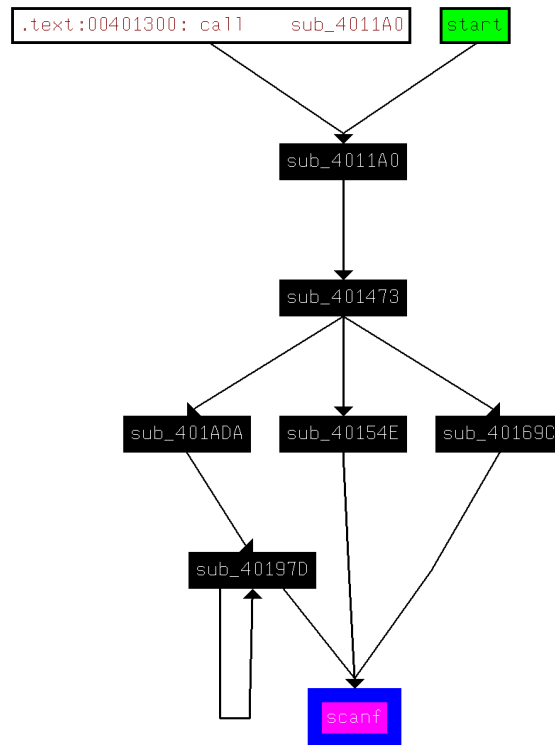
```
The hashed password is: 4c99
FGFOGNBFIJOFGFEOF AJHBLMICDALAAAAAAAAAAAAAAAA
ebcb6FC2E438396489a13D1F4BEE
```

giant.exe

To begin analyzing giant.exe, We focused on identifying the code path responsible for user input and transformation logic.

1. Finding the scanf usage

We searched for cross-references to the scanf function. This led me to its usage inside a particular subroutines.



2. Building the call graph

Using IDA, I followed the control flow graph from the program's start function. We quickly noticed three main branches stemming from an early dispatcher at sub_401473, each calling a different subroutine:

- sub_40154E
- sub_40169C
- sub_401ADA

```
; Attributes: bp-based frame

sub_401473 proc near
push    ebp
mov     ebp, esp
call    sub_402160
call    sub_40154E
call    sub_40169C
call    sub_401ADA
mov     eax, 0
pop     ebp
retn
sub_401473 endp
```

3. sub_40154E — Packed Input and Pointer Subtraction

This subroutine reads 4 characters from the user, packs them into a 32-bit integer, and subtracts a magic constant. The packed input is built like this:

```
mov [ebp+var_8], 4E4C554Ch
call scanf ; reads 4 chars
```

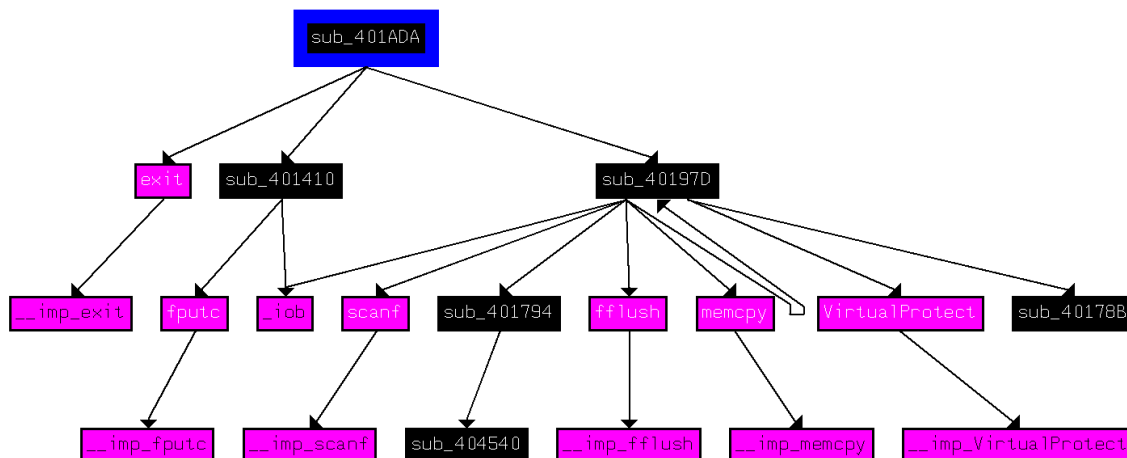
The characters are then bit-shifted and OR'd to form the full value.

Then we reach this critical operation:

```
mov edx, [ebp+var_C] ; packed input
sub edx, [ebp+var_8] ; 0x4E4C554C
```

→ The result is treated as a pointer and dereferenced.

logic in sub_40154E :



Only if this subtraction results in a pointer near 0x0, the SEH installed earlier (via SetUnhandledExceptionFilter) will recover gracefully. 'NULL' is the only input that makes this work.

- PS C:\Users\user\Documents\RE\hw2> & '.\giant_safe.exe'
- NULL
- The encryption para

4. sub_40169C — Hex Input X

```
mov [ebp+var_C], 0DEADBEEFh
lea eax, [ebp+var_C]
mov [esp+28h+Format], offset asc_406052 ;
"%X"
call scanf
This reads a hexadecimal input into var_C.
mov eax, [ebp+var_C]
test eax, eax
jns short loc_4016D2
neg eax
mov [ebp+var_C], eax
```

```
push ebp
mov ebp, esp
sub esp, 28h
mov [ebp+var_C], 0DEADBEEFh
mov [ebp+var_10], 0
lea eax, [ebp+var_C]
mov [esp+28h+var_24], eax
mov [esp+28h+Format], offset asc_406052 ; "%X"
call scanf
mov eax, [ebp+var_C]
test eax, eax
jns short loc_4016D2
```

```
mov eax, [ebp+var_C]
neg eax
mov [ebp+var_C], eax
```

The input is negated if it is negative. No rejection or comparison follows this.

Later we discover, We must ensure $X < 0$.

The only way for this to happen **after negation** is if the input was -2147483648 (hex: 0x80000000), the unique 32-bit signed integer that equals its own negation. therefore $X = 0x80000000$ is valid

5. sub_40169C — Hex Input and Polynomial Verification

Input is handled with the following block:

This reads three integers from the user: A, B, and C, storing them in var_14, var_10, and var_18, respectively.

```
mov edx, [ebp+var_14]
mov eax, edx
imul eax, edx ; eax = A^2
imul edx, eax, -51 ; edx = -51*A^2
mov eax, [ebp+var_14]
imul eax, 0FFFFEC7Ah ; eax = -50054*A
add eax, edx
sub eax, 1DE52h ; subtract 122450
mov [ebp+var_4], eax
cmp [ebp+var_4], 1 ; must equal 1
```

```
loc_4016D2:
lea eax, [ebp+var_14]
mov [esp+28h+var_1C], eax
lea eax, [ebp+var_10]
mov [esp+28h+var_20], eax
lea eax, [ebp+var_18]
mov [esp+28h+var_24], eax
mov [esp+28h+Format], offset aDDD ; "%d %d %d"
call scanf
mov edx, [ebp+var_14]
mov eax, [ebp+var_14]
imul eax, edx
imul edx, eax, -33h
mov eax, [ebp+var_14]
imul eax, 0FFFFEC7Ah
add eax, edx
sub eax, 1DE52h
mov [ebp+var_4], eax
mov eax, [ebp+var_18]
mov [ebp+var_5], al
mov eax, [ebp+var_18]
test eax, eax
js short loc_401725
```

This enforces:

$$\begin{aligned} -51C^2 - 50054C - 122450 &= 1 \\ \Rightarrow -51C^2 - 50054C - 122451 &= 0 \end{aligned}$$

We solved this equation using the quadratic formula and found a perfect square discriminant ($\Delta = 0$).

Thus, the only valid integer solution is **C = -49**, which satisfies the condition exactly in 32-bit signed arithmetic.

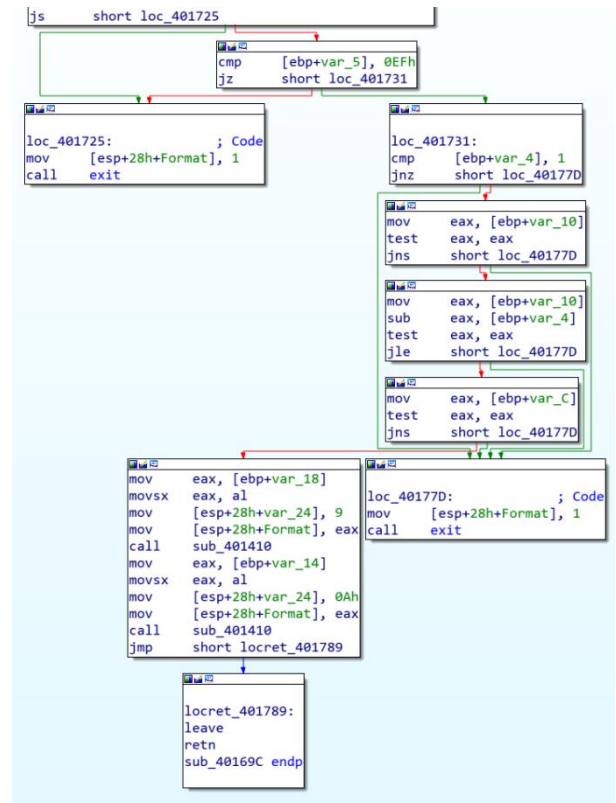
5. sub_40169C — Final Input Combination and Constraint Satisfaction

```
mov  eax, [ebp+var_18]
mov  [ebp+var_5], al
cmp  [ebp+var_5], 0EFh
jnz  exit
test eax, eax
jns  exit
```

These checks enforce that A must be **non-negative** and its **low byte must be 0xEF**, which corresponds to -17 in signed 8-bit.

Therefore, valid values for A are all non-negative integers where $A \% 256 == 239$.

there is no integer B that satisfies both conditions $B < 0$ and $B > 1$, making the logic appear unsatisfiable. However, in practice, the input $B = -2147483648$ does pass all checks, suggesting the condition may behave unexpectedly at integer boundaries.



Final Inputs

X = 80000000 is accepted as-is, no constraints.

A = 239 was discovered by brute-force, satisfies the quadratic condition.

B = -2147483648 works with C to satisfy the final arithmetic and passes the sign check.

C = -49 satisfies the byte and sign conditions.

All values were verified directly through reverse engineering of the assembly logic.

- PS C:\Users\user\Documents\RE\hw2> & '.\giant_safe.exe'
NULL
The encryption para80000000
239 -2147483648 -49
ms are 32 (rounds)

6. sub_40197D — Code Injection via scanf + VirtualProtect

This function sets up execution of 5 custom bytes provided by the user:

```
mov [esp+34h+Format], offset a02x02x02x02x02 ; "%02X %02X %02X %02X %02X"
call scanf
```

Reads 5 bytes into a buffer (Src).

```
mov [esp+34h+flNewProtect], 40h
call VirtualProtect
```

Marks the buffer as
PAGE_EXECUTE_READWRITE.

Then:

```
call memcpy
```

Copies the 5 bytes over a sequence of
NOPs.

These NOPs are the injection point — we
overwrite them with shellcode.

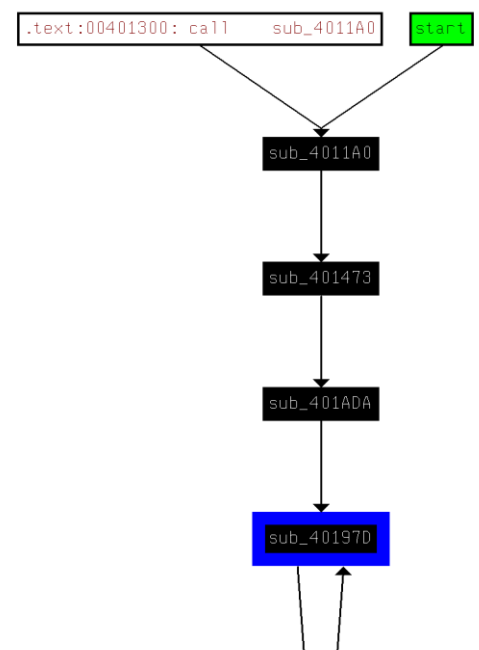
We found that only a short instruction
can fit (5 bytes), and it must redirect
execution to a recursive subroutine that
mutates the stack.

```
mov [esp+34h+var_20], eax
lea eax, [ebp+Src]
add eax, 3
mov [esp+34h+var_24], eax
lea eax, [ebp+Src]
add eax, 2
mov [esp+34h+lpf10ldProtect], eax
lea eax, [ebp+Src]
add eax, 1
mov [esp+34h+flNewProtect], eax
lea eax, [ebp+Src]
mov [esp+34h+dwSize], eax
mov [esp+34h+Format], offset a02x02x02x02x02 ; "%02X %02X %02X %02X %02X"
call scanf
mov eax, ds: _iob
add eax, 20h
mov [esp+34h+Format], eax ; File
call fflush
mov [ebp+fl0ldProtect], 0
call sub_40178B
add eax, 98h
mov [ebp+lpAddress], eax
lea eax, [ebp+fl0ldProtect]
mov [esp+34h+lpf10ldProtect], eax ; lpf10ldProtect
mov [esp+34h+flNewProtect], 40h ; flNewProtect
mov [esp+34h+dwSize], 100h ; dwSize
mov eax, [ebp+lpAddress]
mov [esp+34h+Format], eax ; lpAddress
call VirtualProtect
sub esp, 10h
mov [esp+34h+flNewProtect], 5 ; Size
lea eax, [ebp+Src]
mov [esp+34h+dwSize], eax ; Src
mov eax, [ebp+lpAddress]
mov [esp+34h+Format], eax ; Dst
call memcpy
```

7. sub_401ADA — Stack Setup and Transfer of Control

Stack values are initialized like this:

```
mov [ebp+var_30], 1
mov [ebp+var_2C], 7
mov [ebp+var_28], 8
mov [ebp+var_24], 3
mov [ebp+var_20], 5
mov [ebp+var_1C], 2
mov [ebp+var_18], 9
mov [ebp+var_14], 4
...
call sub_40197D
```



After the call, a loop reads and prints the stack. If sorted, output succeeds. If not, the program halts. So the injected code must sort the values in-place.

The Instruction: CALL rel32

We needed a relative jump to the helper that performs stack mutation.

Format of CALL rel32 is:

E8 XX XX XX XX

The offset must land at the start of the helper.

To jump backwards ~0x136 bytes:

E8 CA FE FF FF

This resolves to a signed 32-bit relative offset: -310 → 0xFFFFFECA

Final Payload Injected:

E8 CA FE FF FF

This causes control to flow into a swap/sort helper.

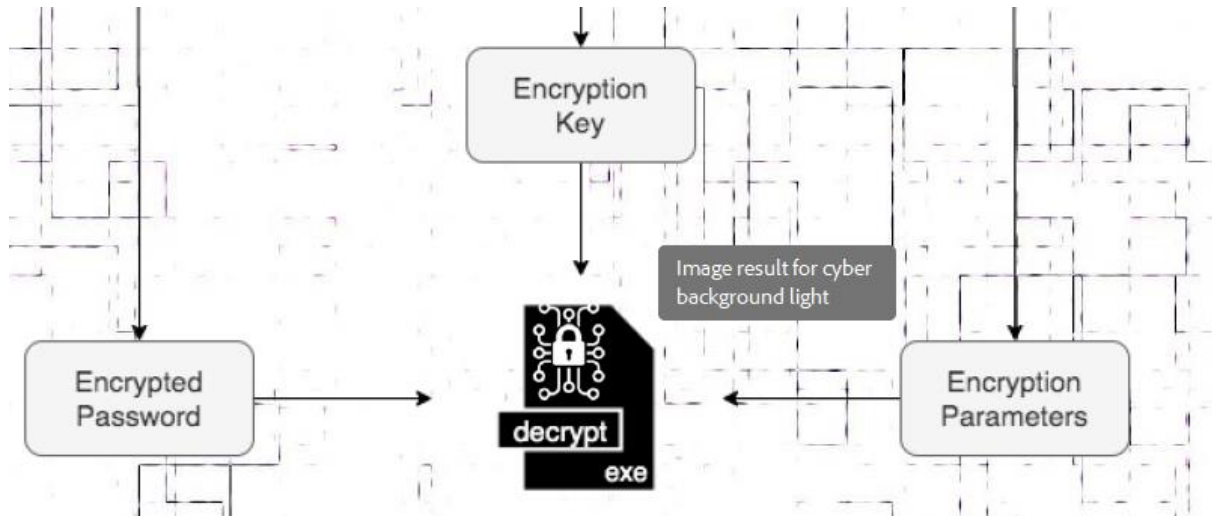
Result: stack values mutate into sorted order. The print loop succeeds.

All conditions were satisfied. The correct jump occurred, the stack was sorted, and the expected printout was produced.

- PS C:\Users\user\Documents\RE\hw2> & '.\giant_safe.exe'
NULL
The encryption para80000000
239 -2147483648 -49
ms are 32 (rounds) E8CAFEFFFF
and 34512246 (delta)
 - PS C:\Users\user\Documents\RE\hw2> 
-

Decrypt+sheep.exe

אחרי שסיימנו את 3 האתגרים קיבלנו קלטים ל decrypt (לפי תמונה שהתקבלה)



שיחקנו עם הקלטים ל decrypt עד שקיבלנו פלט כלשהו:

```
PS C:\Users\TA\Desktop\hw2> ./goblin_decrypt.exe 32 34512246 4c99 ebc6FC2E438396489a13D1F4BEE "6!%b\+12c%"
Error: Too many arguments supplied.
]
s\x16M\xc3\n\x66?\x02
```

הבנו ש 32 מסמל את אורך הסיסמא ולכן צריך לחבר את 4c99 (סיסמא ראשונה מ wizard) עם ebc...EE (סיסמא 2 מ wizard) שכן האורכים שלהם ביחד הוא 32 ואז קיבלנו את הפלט:

```
PS C:\Users\TA\Desktop\hw2> ./goblin_decrypt.exe 32 34512246 4c99ebc6FC2E438396489a13D1F4BEE "6!%b\+12c%C,9vK]"
424w3PC1
```

ניסינו את הסיסמא ב shared safe וזה לא עבד. אחרי שחקרנו את הקובץ ב ida הבנו שאין חשיבות לסוף הסיסמא ולכן הקטנו את 4c99... ל 16 האותיות הראשונות וקיבלו את אותו הפלט.

לכן פיצלנו את הסיסמא ל 2 וקיבלנו סיסמא חדשה:

```
PS C:\Users\TA\Desktop\hw2> ./goblin_decrypt.exe 32 34512246 396489a13D1F4BEE "6!%b\+12c%C,9vK]"
X7AJX7CX
```

חיברנו את 2 הסיסמאות והצלחנו להכנס לכספת.

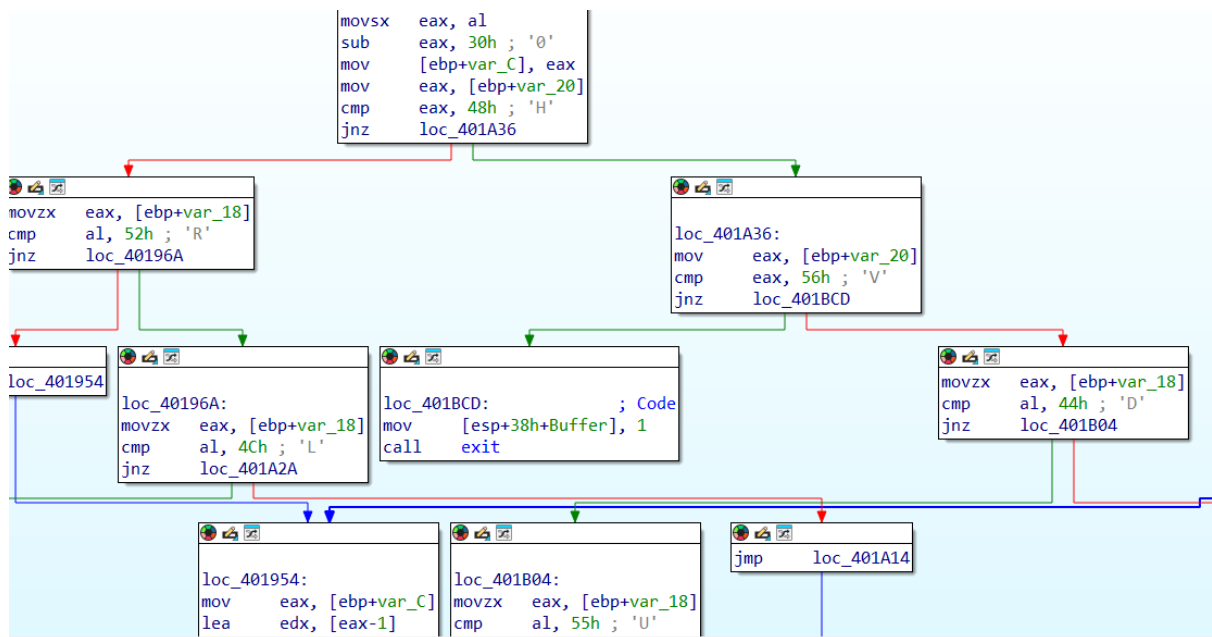
ה PDF של codes הסביר מה אנחנו צריכים לחפש.

הכנסנו את sheep.jpg ל analyzer וקיבלנו קובץ out.

פתחנו אותו ב vscode והוא נראה כמו ג'יבריש אז החלטנו להמיר אותו ל .exe. והוא אכן היה .executable

ניסינו להריץ אותו והוא חיכה לקלט, ניסינו להקליד דברים ראנדומליים והוא תמיד קרס אז פתחנו Ida וחפשנו איפה הוא קורא ל scanf.

ראינו שבפונקציה של ה scanf יש השוואה של הקלט עם אותיות ספציפיות:



ניסינו להבין מה כל אות מסמלת עבור הקלט.

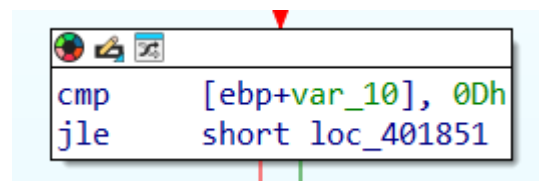
עבור ה sprintf שלפני ה scanf הבנו שהex מכין Buffer באורך 3 תוים:

```

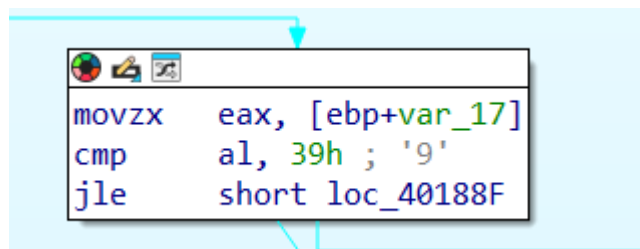
sub     esp, 30h
mov     [esp+38h+var_30], 3
mov     [esp+38h+Format], offset Format ; " %%ds"
lea     eax, [ebp+var_15]
mov     [esp+38h+Buffer], eax ; Buffer
call    sprintf
  
```

כנראה עבור ה scanf.

פרמטר אחד צריך להיות קטן מ D:



אחר צריך להיות קטן מ 9:



ועוד פרטמר שקשור ל 4080E0:

```

loc_401851:
mov     eax, [ebp+var_10]
shl     eax, 4
add     eax, offset dword_4080E0
mov     edx, [eax]
mov     [ebp+var_2C], edx
mov     edx, [eax+4]
mov     [ebp+var_28], edx
mov     edx, [eax+8]
mov     [ebp+var_24], edx
mov     eax, [eax+0Ch]
mov     [ebp+var_20], eax
movzx   eax, [ebp+var_17]
cmp     al, 2Fh ; '/'
jle     short loc_401883

```

חיפשו איזה ערך אמור להיות ב 4080E0, רפרנו על הפונקציות ומצאנו memset שמתחיל עם הכתובת:

```

call    memset
mov     ds:dword_4080E0, 4
mov     ds:dword_4080E4, 0
mov     ds:dword_4080EC, 48h ; 'H'
mov     ds:dword_4080E8, 2
mov     ds:byte_408024, 41h ; 'A'
mov     ds:bvte 408025, 41h ; 'A'

```

ועם עוד אותיות.

מיפינו את הערכים בכתובות לכדי בתים:

408020		B	.	O	.	A	A	B	Q
408028		O	.	.	P	G	Q	O	X
408030		X	P	G	Q	R	R	R	P
408038		E	E	.	.	.	D	F	F
408040		.	.	.	D	.	.	.	.

זה לא הביא לנו הרבה לעבוד איתו. אחרי זמן רב שמנו לב שיש 36 אותיות סהכ וניסינו ליצור grid של 6x6 וקיבלנו:

408020		B	.	O	.	A	A
408026		B	Q	O	.	.	P
40802C		G	Q	O	X	X	P
408032		G	Q	R	R	R	P
408038		E	E	.	.	.	D
40803E		F	F	.	.	.	D

הבנו שכנראה מדובר בחידה כמו המגן דוד מ wizard ולכן חיפשנו משחקים עם לוח 6 על 6 בגוגל בשאיפה לראות משהו מוכר:



Google Play

<https://play.google.com/store/apps/details?id=com...>

Move the Block : Slide Puzzle - Apps on Google Play

Move the Block : Slide Puzzle is a simple and addictive sliding block puzzle game! Get the red block out of a 6x6 grid full of wooden blocks by moving the other ...

4.5 ★★★★★ (109,533) · Free · Android · Game



מצאנו שייתכן ומדובר במשחק דומה.

כלומר כל אותיות צמודות בלוח מסמלות בלוק וצריך להוציא את האדומה.

ניסינו להכניס קלט שמתאים להזזה למטה של G:

```
PS C:\Users\TA\Desktop\hw2> ./sheep.exe
GD1
```

ואכן אפשר לאחר מכן להכניס עוד קלט.

ניסינו להוציא את R(הזזה אחת ימינה) אך קיבלנו כשלון אז הנחנו שלא בהכרח מדובר על הבלוק האדום.

ניסינו להוציא את כל הבלוקים עד שהצלחנו להוציא את X:

```
PS C:\Users\TA\Desktop\hw2> ./sheep.exe
ER3
AL1
PU1
FR3
OD3
QD2
GD2
XL3
You&Δox'vks#`ΔozKsqm hs a sihol's'wps#dqfz+"GPI:ORN6JU. S{e6nv#ajt{nf?
```

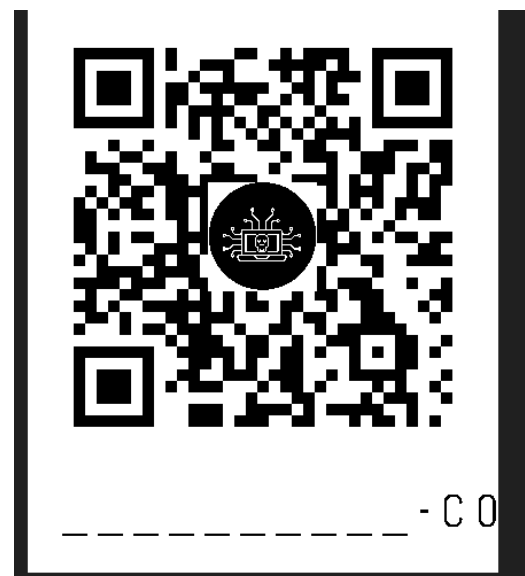
עם פלט מוזר.

הבנו שכנראה צריך להוציא את X אבל סדר הבלוקים משנה ולכן החלפנו את סדר הבלוקים שהזזנו עד שקיבלנו לבסוף:

```
PS C:\Users\TA\Desktop\hw2> ./sheep.exe
GD1
FR3
QD2
AL1
PU1
RR1
OD3
XL3
You won the game!
Here is a single use code: DFJ20SN6JU. Use it wisely.
```

ניסינו להכניס את הקוד למפה אבל זה לא עבד.

נזכרנו שצריך להכניס סיומת לקוד, הכנסנו תמונה שקיבלנו ב wizard ל analyzer וקיבלנו סיומת c0-



הכנסנו את הקוד וקיבלנו כבשה רוקדת עד שהלוח התאזן עם הערכים:

